



ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

Universidad de las Fuerzas Armadas ESPE

Carrera de Software

Desarrollo de Videojuegos

Informe sobre el algoritmo de búsqueda de caminos (BFS) en videojuegos

Proyecto: BFS aplicado a Bad Ice Cream

Grupo: 1

Integrantes:

Jordan Guaman

Mesias Mariscal

Anthony Morales

Denise Rea

NRC: 30817

Docente: Ing. Héctor Pinto

Sangolquí, Ecuador

Junio de 2026

G1

Índice

1. Introducción	3
2. Objetivos del informe	4
2.1. Objetivo general	4
2.2. Objetivos específicos	4
3. Técnicas de búsqueda de caminos en videojuegos	5
4. Explicación del algoritmo BFS	7
4.1. Objetivo del algoritmo	7
4.2. Los nodos	7
4.3. Los costos	7
4.4. Los obstáculos	8
4.5. Pasos de ejecución (el motor lógico)	8
4.6. La ruta final (backtracking)	8
4.7. Limitaciones	9
5. Aplicación práctica del algoritmo en un motor de juego	9
5.1. El mapa: tilemap y matrices	9
5.2. Origen y destino	10
5.3. Obstáculos estáticos y dinámicos	10
5.4. Cálculo: el game loop y la cola FIFO	11
5.5. Seguimiento de ruta (path following)	11
6. Caso de estudio: BFS en el videojuego Bad Ice Cream	12
6.1. Personaje analizado	12
6.2. Situación de juego (core loop)	12
6.3. Beneficio del algoritmo para la jugabilidad	13
7. Flujo de ejecución y comparación del algoritmo BFS	14
7.1. Flujo de ejecución	14
7.2. Comparación con otros algoritmos	15
8. Conclusiones	16

1. Introducción

Desde los primeros videojuegos en dos dimensiones, el movimiento de los personajes ha sido uno de los problemas centrales del diseño y la programación. Un enemigo que patrulla sin rumbo, un jugador que no sabe hacia dónde ir o un personaje que se queda atascado contra una pared rompen la inmersión y arruinan la experiencia de juego. Para que el movimiento resulte creíble, funcional y coherente con el entorno, los desarrolladores recurren a un conjunto de técnicas conocidas como **técnicas de movimiento**, dentro de las cuales destaca la **búsqueda de caminos** (*pathfinding*), una de las ramas más importantes de la inteligencia artificial aplicada a videojuegos.

La búsqueda de caminos consiste en calcular automáticamente una ruta válida —y, en la mayoría de los casos, óptima— entre un punto de origen y un punto de destino dentro de un entorno con obstáculos. Esta disciplina agrupa múltiples algoritmos, cada uno con sus propias ventajas y limitaciones: desde los más sencillos como la búsqueda en anchura (*BFS*) y la búsqueda en profundidad (*DFS*), hasta alternativas más elaboradas como Dijkstra, A*, los Campos de Flujo o las variantes jerárquicas. La elección del algoritmo depende del género del juego, del tamaño del mapa, de la cantidad de agentes en movimiento y de si el terreno presenta costos uniformes o variables.

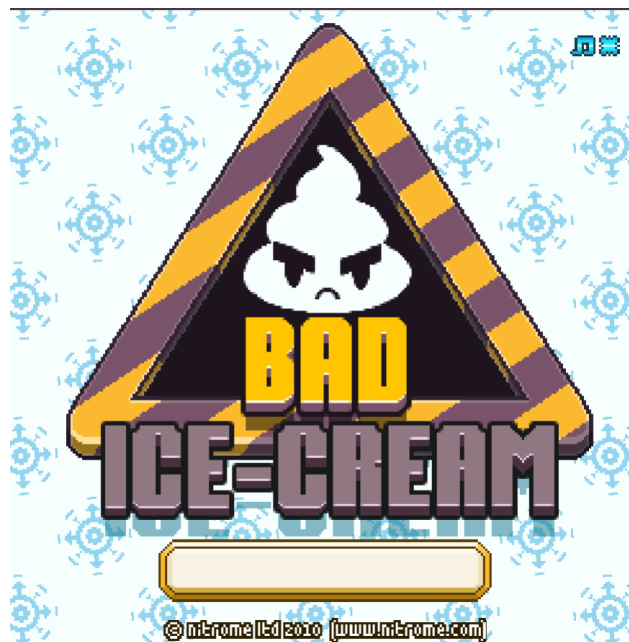


Figura 1: *Bad Ice Cream* (nitrome, 2010): portada oficial del juego que sirve como caso de estudio para el algoritmo *BFS*.

El presente informe se enfoca en uno de esos algoritmos: la búsqueda en anchura (*BFS*, *Breadth-First Search*). Aunque no es el más avanzado, *BFS* destaca por su simplicidad, su optimalidad garantizada en mapas de costos uniformes y su facilidad de implementación, lo que lo convierte en la

opción didáctica y técnicamente adecuada para juegos bidimensionales basados en cuadrícula. Para ilustrar su aplicación real, se analiza el caso del videojuego *Bad Ice Cream*, un título de tipo *arcade* de 2010 en el que los enemigos persiguen al jugador a través de un laberinto de bloques de hielo que se crean y destruyen en tiempo real. Este escenario resulta ideal porque el mapa es una cuadrícula discreta, los costos de movimiento son uniformes y los obstáculos cambian constantemente, lo que obliga a recalcular rutas con frecuencia.

El documento se organiza en cuatro ejes: una investigación de las principales técnicas de movimiento y algoritmos de búsqueda de caminos utilizados en la industria, una explicación teórica detallada del algoritmo *BFS*, una descripción de cómo se implementa dentro del motor de un juego y un análisis del caso real de *Bad Ice Cream* donde se identifican los personajes controlados, las situaciones de juego y los beneficios del algoritmo para la jugabilidad.

2. Objetivos del informe

2.1. Objetivo general

Investigar las técnicas de movimiento aplicadas al desarrollo de videojuegos, exponer el funcionamiento y la aplicación de un algoritmo de búsqueda de caminos, y presentar un juego concreto donde dicho algoritmo se encuentre implementado, tomando como caso de estudio *Bad Ice Cream*.

2.2. Objetivos específicos

- Investigar y comparar las principales técnicas de movimiento y los algoritmos de búsqueda de caminos empleados en la industria del videojuego, identificando la utilidad y las limitaciones de cada uno.
- Explicar de forma detallada el algoritmo *BFS*: su objetivo, el manejo de nodos y costos, el tratamiento de obstáculos, los pasos de ejecución, la reconstrucción de la ruta final y sus limitaciones.
- Describir cómo se implementa *BFS* dentro del motor de un juego: representación del mapa, definición del origen y el destino, manejo de obstáculos dinámicos, ciclo de cálculo y seguimiento de la ruta por parte del personaje.
- Analizar el caso del videojuego *Bad Ice Cream*, identificando los personajes controlados por la IA, las situaciones de juego y el beneficio concreto que *BFS* aporta a la jugabilidad.

3. Técnicas de búsqueda de caminos en videojuegos

La búsqueda de caminos no es un problema único: existen múltiples algoritmos que priorizan distintos criterios, como la optimalidad de la ruta, el uso de memoria o la velocidad de respuesta ante mapas que cambian. Elegir el algoritmo correcto depende del género del juego, del tamaño del mapa, de la cantidad de agentes que se mueven y de si el terreno es uniforme o tiene distintos costos de tránsito.

Tabla 1: Técnicas de búsqueda de caminos y su utilidad en videojuegos

Técnica	Descripción y utilidad en videojuegos
A* (A-Estrella)	Combina el costo real recorrido con una estimación heurística hacia la meta. Es el estándar de la industria; se utiliza en <i>RTS</i> , <i>MOBAs</i> y <i>RPGs</i> para un movimiento preciso y eficiente al hacer clic sobre el terreno.
Dijkstra	Explora en todas las direcciones calculando el costo real hacia cada nodo posible, sin usar heurísticas. Es ideal cuando se busca el punto de interés más cercano (por ejemplo, un botiquín o una zona de cobertura) en lugar de un destino único.
Búsqueda en Anchura (BFS)	Se expande en forma de anillos concéntricos, asumiendo que todos los movimientos tienen el mismo costo. Es excelente para mapas de cuadrícula, como en los juegos de rol tácticos por turnos, para mostrar el área de movimiento posible.
Búsqueda en Profundidad (DFS)	Avanza lo más lejos posible por una ruta y retrocede si encuentra un callejón sin salida. No se usa para mover personajes; su fuerte es la generación procedimental y la verificación de laberintos.
D* y LPA*	Variantes avanzadas de A* que almacenan cálculos previos para adaptarse a cambios sin reiniciar. Son indispensables en entornos dinámicos y destructibles donde los obstáculos cambian repentinamente en medio del camino.
Puntos de Salto (JPS)	Optimización matemática de A* que se salta nodos vacíos y busca directamente cambios de dirección forzados. Es fundamental en simuladores y <i>RTS</i> a gran escala para mover miles de unidades simultáneamente sin saturar la CPU.
Voraz el Mejor Primero	Ignora el costo recorrido y se guía exclusivamente por la estimación visual hacia el objetivo. Es perfecto para hordas de enemigos "tontos" agresivos (como zombis) que persiguen al jugador sin buscar la ruta perfecta.
Theta*	Permite el cálculo de trayectorias en línea recta entre dos puntos si no hay obstáculos en medio. Se usa en juegos modernos 3D y mundos abiertos para evitar movimientos robóticos en zigzag sobre cuadrículas invisibles.
Jerárquico A* (HPA*)	Divide un mapa gigante en áreas más pequeñas y calcula rutas abstractas de un área a otra antes del detalle. Es crucial en juegos de mundo abierto (<i>MMORPGs</i>) para que la IA planee viajes largos sin consumir enormes cantidades de memoria.
Campos de Flujo	Asigna un vector direccional a cada celda del mapa, guiando el movimiento como si fuera una corriente. Es la solución definitiva para juegos con enjambres masivos, permitiendo a cientos de unidades navegar sin calcular cientos de rutas individuales.

De la tabla anterior se observa que *BFS* no es el algoritmo más avanzado ni el más eficiente en mapas grandes, pero sí es el más sencillo de implementar y el más confiable cuando el terreno es uniforme y el mapa se representa como una matriz discreta. Esa es la razón por la que resulta perfecto para juegos como *Bad Ice Cream*, donde el escenario es una cuadrícula, los costos de movimiento son iguales para todas las celdas y los enemigos necesitan recalcular su ruta cada vez que el jugador modifica el entorno.

4. Explicación del algoritmo BFS

4.1. Objetivo del algoritmo

El objetivo principal de *BFS* es encontrar el camino más corto garantizado entre un punto de origen y un destino dentro de un grafo no ponderado, como por ejemplo una cuadrícula plana. Su filosofía es explorar el entorno por "capas" o "anillos concéntricos": primero revisa todos los lugares que están a un paso de distancia, después los que están a dos pasos, y así sucesivamente, hasta localizar el destino.

4.2. Los nodos

En el contexto de un videojuego en 2D, el mapa se divide en una cuadrícula (matriz).

- Cada celda o casilla de esa cuadrícula es un **nodo**.
- Cada nodo almacena información vital: sus coordenadas (x, y) , su estado (*transitable* o *no transitable*) y, durante la ejecución del algoritmo, un puntero hacia su **nodo padre**, es decir, la casilla desde la cual el algoritmo llegó a él.

4.3. Los costos

A diferencia del algoritmo de Dijkstra o de A^* , *BFS* no maneja costos variables.

- Asume que moverse a cualquier nodo vecino cuesta exactamente lo mismo, es decir, **costo = 1**.
- No distingue entre caminar por asfalto, lodo o agua. Por esta razón, solo es útil en juegos donde el terreno es uniforme y el único factor que importa es la cantidad de pasos (o casillas) para llegar al objetivo.

4.4. Los obstáculos

Los obstáculos (paredes, bloques de hielo, enemigos) son simplemente nodos marcados lógicamente como *no transitables* o bloqueados. Cuando el algoritmo evalúa los nodos vecinos para expandirse, verifica esta propiedad: si el nodo es un obstáculo, lo descarta de inmediato y no lo añade a su lista de exploración. Esto obliga al .anillo de búsqueda.a rodear el bloqueo y continuar la exploración por los caminos válidos.

4.5. Pasos de ejecución (el motor lógico)

Para implementarlo, *BFS* utiliza una estructura de datos llamada **Cola** (*Queue*), que funciona bajo el principio *FIFO* (*First In, First Out*: el primero en entrar es el primero en salir). El algoritmo sigue esta secuencia:

1. **Inicialización:** se crea la Cola de exploración y una lista de nodos *Visitados*, cuyo propósito es no procesar la misma casilla dos veces.
2. **Arranque:** se toma el nodo de Origen, se marca como *Visitado* y se encola.
3. **Ciclo de búsqueda:** mientras la Cola no esté vacía, se desencola (extrae) el primer nodo.
4. **Validación de destino:** si el nodo extraído es el Destino, el algoritmo se detiene (es un éxito).
5. **Expansión:** si no es el destino, el algoritmo revisa sus nodos vecinos inmediatos: arriba, abajo, izquierda y derecha.
6. **Encolado:** por cada vecino que sea transitable y que no haya sido visitado antes:
 - Se marca como *Visitado*.
 - Se guarda el nodo actual como su *padre*.
 - Se añade al final de la Cola.
7. El ciclo se repite desde el paso 3.

4.6. La ruta final (backtracking)

Una vez que el algoritmo "toca" el nodo Destino en el paso 4, la búsqueda termina, pero aún no se tiene la ruta completa. Para reconstruirla, se realiza un proceso de **rastreo inverso**:

1. El algoritmo toma el nodo Destino y lee quién es su nodo padre.

2. Salta a ese padre y lee al abuelo, luego al bisabuelo, y así sucesivamente.
3. Continúa este rastreo hacia atrás hasta llegar al nodo de Origen.
4. Finalmente, invierte esta lista de nodos, entregando al personaje la secuencia exacta de pasos hacia adelante para llegar a la meta.

4.7. Limitaciones

Es importante mencionar por qué *BFS* no se utiliza en todos los juegos:

- **Alto consumo de memoria (RAM):** al expandirse en todas las direcciones de forma indiscriminada (como una gota de agua cayendo en un charco), guarda demasiados nodos inútiles en la memoria antes de encontrar el destino.
- **Ceguera direccional (sin heurística):** no sabe hacia dónde está la meta. Si el destino está a la derecha, *BFS* igual perderá tiempo explorando a la izquierda, arriba y abajo.
- **Inútil en terrenos complejos:** como no evalúa costos, si hay un camino corto pero lleno de lava y un camino ligeramente más largo pero seguro, *BFS* elegirá ciegamente el de la lava por tener menos pasos.

5. Aplicación práctica del algoritmo en un motor de juego

En el desarrollo de un juego como *Bad Ice Cream*, el algoritmo no flota en el aire: interactúa directamente con la estructura de datos del motor. A continuación se describe cómo funciona *BFS* "bajo el capó".

5.1. El mapa: tilemap y matrices

El escenario completo no es más que una matriz bidimensional (un *array* 2D) en la memoria del juego. Cada fruta, muro de hielo o espacio vacío ocupa una celda exacta, por ejemplo `mapa[x][y]`. Esto simplifica enormemente el entorno para el algoritmo, ya que convierte un mundo gráfico en un grafo matemático perfecto.



Figura 2: Cuadrícula completa de *Bad Ice Cream*: el escenario se modela como una matriz mapa [x] [y] donde cada celda es un nodo para *BFS*.

5.2. Origen y destino

- El **Origen** se actualiza constantemente y corresponde a las coordenadas actuales en la matriz del enemigo (por ejemplo, la Vaca o el Monstruo Verde).
- El **Destino** son las coordenadas dinámicas del jugador, es decir, el helado que el enemigo intenta atrapar.

5.3. Obstáculos estáticos y dinámicos

A nivel lógico, una celda vacía tiene un valor de 0 (transitable) y un bloque de hielo tiene un valor de 1 (no transitable). La particularidad de este juego es que el mapa **muta en tiempo real**: cuando el jugador dispara su habilidad, cambia los valores de la matriz de 0 a 1 (creando hielo) o de 1 a 0 (rompiendo hielo). Esta naturaleza dinámica obliga a recalcular las rutas de los enemigos constantemente.



Figura 3: Obstáculos dinámicos: el jugador rompe bloques de hielo (cambiando celdas de 1 a 0), lo que obliga a la IA a recalcular la ruta con *BFS*.

5.4. Cálculo: el game loop y la cola FIFO

- Para no saturar el procesador, *BFS* no se calcula en cada fotograma (*frame*), sino a través de eventos (*event-driven*): se recalcula cada vez que el mapa cambia (cuando el jugador crea o rompe hielo) o cuando el enemigo termina de moverse a una casilla.
- El algoritmo utiliza una estructura de datos de Cola (*FIFO* - *First In, First Out*). Inserta la posición del enemigo y comienza a encolar las casillas vecinas (Norte, Sur, Este, Oeste) que tengan valor 0, descartando las que tengan valor 1. Se expande en forma de onda hasta que las coordenadas evaluadas coincidan con las del jugador.

5.5. Seguimiento de ruta (path following)

Una vez que *BFS* alcanza al jugador, realiza un rastreo inverso a través de los nodos padre para construir un arreglo (array) unidimensional con los pasos exactos, por ejemplo [Arriba, Arriba, Izquierda, Abajo]. El script de movimiento del enemigo simplemente lee el primer elemento de ese arreglo, mueve el *sprite* gráficamente a esa celda, lo elimina del arreglo y pasa al siguiente elemento. De esta forma, el enemigo avanza casilla por casilla siguiendo la ruta calculada.

6. Caso de estudio: BFS en el videojuego Bad Ice Cream

6.1. Personaje analizado

El **personaje analizado** son los enemigos controlados por la IA (*Inteligencia Artificial*). En los primeros niveles, algunos monstruos solo patrullan de forma aleatoria, pero los enemigos avanzados entran en un estado de **persecución activa** donde entra en juego el algoritmo *BFS*. Estos enemigos son los responsables de dar tensión al juego, ya que están diseñados para alcanzar al jugador a través del laberinto, sin importar cuántas paredes de hielo se coloquen en su camino.



Figura 4: Personajes controlados por la IA: los enemigos *slime* persiguen al helado activando el modo de persecución mediante *BFS*.

6.2. Situación de juego (core loop)

La mecánica principal (*core mechanic*) de *Bad Ice Cream* es el **control del espacio**. El jugador está atrapado en una arena cerrada recogiendo frutas, y su única defensa es alterar la arquitectura del laberinto construyendo o destruyendo paredes de hielo, ya sea para encerrar a los enemigos o para abrirse paso y escapar. Cada decisión espacial del jugador modifica la matriz del mapa, lo que obliga a la IA a recalcular su ruta de inmediato.



Figura 5: Core loop: el jugador modifica el laberinto de hielo mientras múltiples tipos de enemigos (yetis, cascos, fuego) lo persiguen.

6.3. Beneficio del algoritmo para la jugabilidad

El uso de *BFS* en *Bad Ice Cream* aporta dos beneficios claros a la experiencia de juego:

- **Evita el "Queso"(Cheesing):** en diseño de videojuegos, *cheesear* significa usar un truco barato para ganar sin esfuerzo. Si los enemigos no tuvieran *BFS* y solo caminaran hacia el jugador en línea recta, el jugador podría simplemente poner un bloque de hielo frente a ellos y quedarse quieto recogiendo frutas, mientras el enemigo choca infinitamente contra el hielo. *BFS* elimina esta estrategia porque el enemigo siempre busca un camino alternativo.
- **Presión constante:** gracias a *BFS*, en el instante en que el jugador coloca un muro de hielo, la matriz se actualiza y la IA percibe que su camino actual está bloqueado. El algoritmo ejecuta una nueva búsqueda en anchura, encuentra el pasillo lateral más cercano y flanquea al jugador. Esto obliga al jugador a estar en constante movimiento y a pensar rápido, creando la dificultad y el valor de entretenimiento reales del juego.

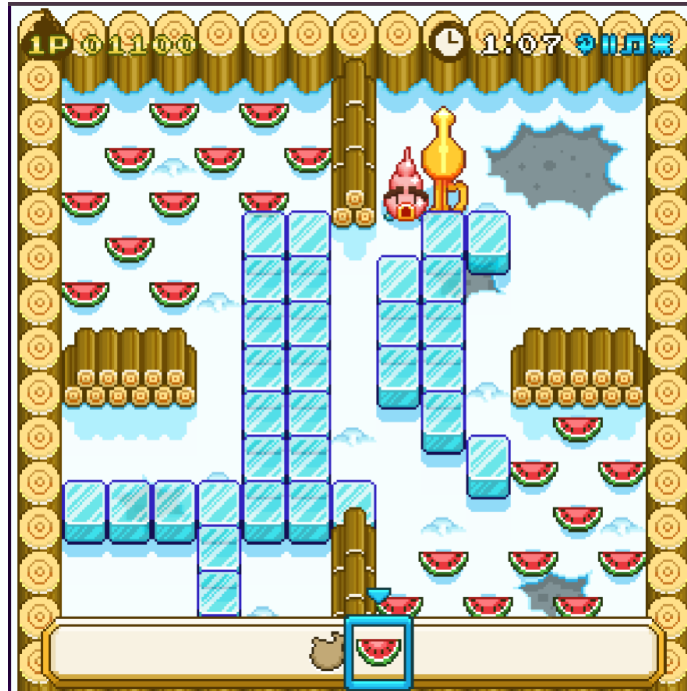


Figura 6: Presión constante: el enemigo de fuego navega una estructura compleja de paredes de hielo, rodeando los muros para alcanzar al jugador.

7. Flujo de ejecución y comparación del algoritmo BFS

7.1. Flujo de ejecución

El siguiente diagrama resume el flujo lógico que sigue *BFS* desde su inicialización hasta la reconstrucción de la ruta final mediante *backtracking*.

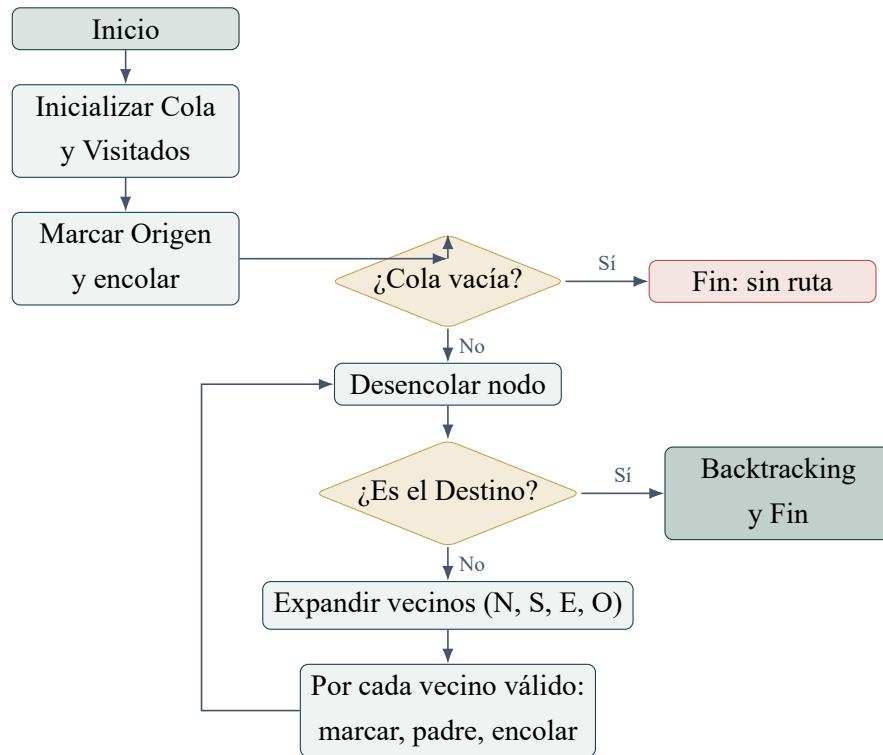


Figura 7: Flujo de ejecución del algoritmo *BFS*: inicialización (izquierda) y bucle principal (derecha) con retorno al desencolar.

7.2. Comparación con otros algoritmos

Para situar a *BFS* frente a sus alternativas más comunes, la siguiente tabla resume sus características frente a Dijkstra y A^* .

Tabla 2: Comparación entre BFS, Dijkstra y A*

Criterio	BFS	Dijkstra	A*
Tipo de grafo	No ponderado	Ponderado (costos ≥ 0)	Ponderado con heurística.
Usa heurística	No	No	Sí.
Camino más corto	Sí (en pasos)	Sí (en costo)	Sí (si la heurística es admisible).
Complejidad temporal	$O(V + E)$	$O((V + E) \log V)$	$O((V + E) \log V)$ en el mejor caso.
Consumo de memoria	Alto en mapas grandes	Alto en mapas grandes	Menor que BFS gracias a la heurística.
Terreno uniforme	Ideal	No aporta ventaja	No aporta ventaja.
Terreno con costos	No recomendado	Ideal	Ideal.
Mapa cambiante	Recalcula completo	Recalcula completo	Recalcula; mejor con D*/LPA*.
Caso ideal	<i>Bad Ice Cream</i> , juegos por turnos.	Botiquín más cercano, cobertura.	Movimiento con clic en <i>RTS/RPGs</i> .

La tabla anterior confirma que *BFS* no compite en mapas grandes con costos variables, pero sigue siendo la elección natural en el nicho específico de los juegos bidimensionales de cuadrícula con terreno uniforme y obstáculos que cambian a alta frecuencia, como es el caso de *Bad Ice Cream*.

8. Conclusiones

El análisis realizado confirma que el algoritmo de búsqueda en anchura (*BFS*) es una herramienta fundamental dentro del desarrollo de videojuegos bidimensionales, no por ser el más sofisticado, sino por ser el más adecuado cuando el mapa se representa como una cuadrícula de costos uniformes.

En primer lugar, la investigación de las distintas técnicas de búsqueda de caminos permitió identificar el nicho exacto donde *BFS* ofrece su mayor ventaja: mapas de cuadrícula pequeños o medianos con terreno homogéneo, donde se necesita un camino óptimo en pasos y no en costo variable. Frente a algoritmos más avanzados como A*, JPS o los Campos de Flujo, *BFS* se posiciona como la opción más simple, más fácil de implementar y suficiente para juegos de tipo *arcade*.

En segundo lugar, la explicación teórica del algoritmo demostró que su lógica es elegante y

compacta: inicializar, encolar, expandir en capas y reconstruir con *backtracking*. Sin embargo, también quedaron claras sus limitaciones: alto consumo de memoria en mapas grandes, ausencia de heurística que guíe la búsqueda y ceguera frente a terrenos con distintos costos. Estas limitaciones explican por qué la industria reserva *BFS* para casos muy concretos, como los juegos por turnos y los *arcade* de cuadrícula.

En tercer lugar, la aplicación práctica en el motor del juego evidenció que la potencia de *BFS* no está solo en encontrar la ruta, sino en poder recalcularla de manera continua. En *Bad Ice Cream*, el mapa muta cada vez que el jugador dispara su poder, y aun así el algoritmo responde con suficiente rapidez para mantener la persecución fluida. La estructura de Cola (*FIFO*) encaja perfectamente con un motor basado en eventos (*event-driven*) y permite que cada enemigo tenga su propia ruta independiente.

Finalmente, el caso de estudio de *Bad Ice Cream* validó la utilidad de *BFS* para la jugabilidad. Sin *BFS*, los enemigos podrían ser “engañados” fácilmente colocando un bloque de hielo; con *BFS*, el enemigo siempre encuentra un camino alternativo, lo que elimina el *cheese* y genera la presión constante que da sentido al reto del juego. Se concluye entonces que, para proyectos de desarrollo de videojuegos con mecánicas similares, *BFS* sigue siendo una solución vigente, didáctica y perfectamente funcional.